

 Review the assignment due date

[_\(https://classroom.github.com/a/htEQkWQg\)](https://classroom.github.com/a/htEQkWQg)

Arquitectura y Organización de Computadoras - DC - UBA

Segundo Parcial - Primer Cuatrimestre de 2025

Normas generales

- El parcial es INDIVIDUAL
- Puede disponer de la bibliografía de la materia y acceder al repositorio de código del taller de system programming, desarrollado durante la cursada
- Pueden acceder a internet para buscar información, pero no está permitida la interacción con chat bots (ni chats con personas). Pueden usar google para acceder al material de referencia, pero para consultas esperamos que utilicen otro buscador (duckduckgo con assist apagado, por ejemplo) ya que las respuestas automáticas con IA de google se encuentran comprendidas en las restricciones indicadas.
- Las resoluciones que incluyan código, pueden usar assembly o C. No es necesario que el código compile correctamente, pero debe tener un nivel de detalle adecuado para lo pedido por el ejercicio.

Régimen de Aprobación

En este parcial se evalúa el manejo de los mecanismos de:

- Manejo de memoria mediante Segmentación y Paginación
- Excepciones, interrupciones de hardware y syscalls mediante Interrupciones al procesador
- Permisos y cambios de nivel de privilegio
- Administración del tiempo de procesamiento y del espacio de memoria para la ejecución de tareas.

Para la Arquitectura Intel x86. Cualquier **error de concepto grave** sobre el funcionamiento de estos mecanismos calificará la entrega como insuficiente. Es decir, la entrega debe demostrar, en la medida exigida por el enunciado, comprensión clara de la mayoría estos mecanismos, e incomprensión de ninguno.

NOTA: Lea el enunciado del parcial hasta el final, antes de comenzar a resolverlo.

Régimen de Promoción

Quien demuestre comprensión clara de **todos** los temas, en la medida exigida por el enunciado, y conteste correctamente las preguntas teóricas encontradas al final de este enunciado, podrá acceder al régimen de promoción.

Esto es válido únicamente para la primer instancia del parcial y no para su recuperatorio.

Modalidad de Entrega

Deberán crear una nueva branch en este repositorio y allí desarrollar su solución en el archivo `Resolucion.md`, en formato Markdown (Acá tienen un [machete](https://github.com/adam-p/markdown-here/wiki/markdown-cheatsheet) (<https://github.com/adam-p/markdown-here/wiki/markdown-cheatsheet>) de cómo dar formato en markdown). Es importante que no utilicen otro formato ya que dificultaría la corrección. Si su solución utiliza código modificado del Trabajo Práctico, es importante que lo incluyan además de explicar en palabras las modificaciones (pueden copiar y pegar cualquier cosa de sus repositorios de TP). También pueden incluir imágenes si así lo desean. Una vez finalizado, completarán la entrega creando un Pull Request e incluyendo a ayoc-bot como reviewer.

Enunciado

En esta oportunidad, nos encargaron diseñar un sistema de asignación de memoria para nuestro kernel, que permita que cada tarea pueda pedir memoria de forma dinámica y liberarla en caso de que ya no la necesite. Como mecanismo de asignación de memoria, el kernel implementará un sistema de *lazy allocation* que permite que las tareas puedan pedir memoria, pero que el kernel no la reserve hasta que realmente se acceda a dicha memoria.

Diseñaremos una syscall `malloco` que permita a las tareas reservar memoria de forma dinámica. Esta syscall recibirá como parámetro la cantidad de memoria a reservar en bytes y devolverá la dirección virtual a partir de la cual se reservó la memoria. Si no hay suficiente memoria disponible, la syscall deberá devolver `NULL`.

```
void* malloco(size_t size);
```

Las condiciones de la asignación de memoria son las siguientes:

- Como máximo, una tarea puede tener asignados hasta 4 MB de memoria total. Si intenta reservar más memoria, la syscall deberá devolver `NULL`.
- El área de memoria virtual reservable empieza en la dirección `0xA10C0000`
- Cada tarea puede pedir varias veces memoria, pero no puede reservar más de 4 MB en total.
- La syscall `malloco` asignará direcciones posteriores al último bloque reservado por la tarea. Si no encuentra memoria virtual suficiente allí, retornará `NULL`. No se reordenarán bloques ni se buscará espacio en direcciones anteriores al último bloque.
- La reserva, desde el punto de vista de la tarea es transparente y contigua, es decir, la tarea no debe preocuparse por la fragmentación de memoria física. El kernel se encargará de asignar las páginas de memoria física a las

direcciones virtuales correspondientes. El comportamiento en caso de que se agote la memoria física es indefinido.

El sistema llevará registro de las reservas de memoria hechas mediante esta syscall. Para ello utilizará un array alojado estáticamente en la memoria del kernel dónde cada elemento representa una reserva. **La definición de este array y sus elementos es parte de la solución del ejercicio.** No nos vamos a preocupar por el tamaño del array, asumiremos que tendrá espacio suficiente para satisfacer las necesidades del sistema. Como se implementará un sistema de *lazy allocation*, el kernel no asignará memoria física hasta que la tarea intente acceder, ya sea por escritura o lectura, a la memoria reservada. En el momento del acceso, si la dirección virtual corresponde a las reservadas por la tarea, el kernel deberá asignar memoria física a la dirección virtual que corresponda. La asignación es gradual, es decir, solamente se asignará una única página física por cada acceso a la memoria reservada. A medida que haya más accesos, se irán asignando más páginas físicas.

- Si el acceso es incorrecto, porque la tarea está leyendo una dirección que no le corresponde, el kernel debe remover la tarea del scheduler, marcar la memoria reservada por la misma para que sea liberada y saltar a la próxima tarea.
- Las páginas de memoria física son obtenidas del area libre de tareas.
- La memoria asignada por este mecanismo debe estar inicializada a cero (como cuando se reserva memoria con 'calloc').

Cuando una tarea termina de usar la memoria reservada, podrá liberarla. Para esto, se implementará una syscall `chau` que recibirá como parámetro la dirección virtual más baja de la memoria reservada.

```
void chau(void* ptr);
```

Esta syscall debe ser asíncrona, es decir, no hará la liberación de forma inmediata, sino que marcará la memoria reservada como "en desuso". Luego, una tarea especial de nivel 0 se encargará de liberar todos los bloques de memoria marcados en desuso. Esta deberá ejecutarse cada 100 ticks de reloj (se puede decir que su comportamiento es similar al de un `garbage collector`).

- Si se pasa un puntero que no fue asignado por la syscall `malloco`, el comportamiento de la syscall `chau` es indefinido.
- Si se pasa un puntero que ya fue liberado, la syscall `chau` no hará nada.
- Si se pasa un puntero que pertenece a un bloque reservado pero no es la dirección más baja, el comportamiento de la syscall `chau` es indefinido.
- Si la tarea continúa usando la memoria una vez liberada, el comportamiento del sistema es indefinido.
- No nos preocuparemos por reciclar la memoria liberada, bastará con liberarla

Se pide:

- Definir el array que llevará registro de las reservas y liberaciones de memoria
- Implementar la syscall `malloco` y el mecanismo completo de *lazy allocation*
- Implementar la syscall `chau` y la tarea especial de nivel 0
- Detallar todas las modificaciones que se hagan al kernel para implementar estos mecanismos

A tener en cuenta para la entrega:

- Indicar **todas las estructuras de sistema** que deben ser modificadas para implementar las soluciones.
- Está permitido utilizar las funciones desarrolladas en las guías y TPs.
- Es necesario que se incluya **una explicación con sus palabras** de la idea general de las soluciones.
- Es necesario explicitar todas las asunciones que hagan sobre el sistema.
- Es necesaria la entrega de código que implemente las soluciones.

Preguntas teóricas

Las siguientes preguntas se corrigen aparte, por favor, entregar sus respuestas en el archivo `RtasTeoricas.md`.

1. A continuación se tienen las entradas de una TLB correspondientes a una misma tarea que ejecuta en un sistema basado en un procesador x86. Las entradas están en el orden en el que se han ido generando las traducciones.

Nro.Pag Descriptor Ctrl

1	7C047	0EF00121	ccc
2	7EEF0	1EF01067	ccc
3	EC004	001F0163	ccc
4	EC005	001F7123	ccc
5	46104	1F011005	ccc
6	46109	1F010027	ccc

Para dicha tarea el registro **CR3 = 0x000E4000** y las tablas de página correspondientes a las direcciones en uso se alojan en páginas de memoria física inmediatamente contiguas al **DTP** en el orden en el que se han ido requiriendo. A modo de ejemplo: la tabla de páginas correspondiente a la entrada Nro 1 de la **TLB** se ubica en la página de memoria contigua a la del DTP, la Nro 2 se aloja en la siguiente página física, y así sucesivamente. Se pide:

- a. Para las entradas 1 y 2 de la TLB, escribir el contenido de sus correspondientes descriptores en cada nivel de la jerarquía de tablas de traducción a direcciones físicas, considerando que se trata de las dos primeras páginas requeridas al ejecutar la tarea.
 - b. Especificar para cada **PDE** que valor deben tener los bits **U/S** y **R/W** para ser consistentes con el contenido de la TLB. Respuestas posibles en cada caso: 0, 1 o X (indistinto).
 - c. Suponiendo que las seis entradas están siendo utilizadas por una misma tarea y por cada task switch se modifica CR3. ¿Qué ocurre con cada una al momento del task switch? ¿Cual es el tratamiento para aquellas que se han modificado?
 - d. ¿Cuál es la función dentro de la TLB de los bits identificados genéricamente como **Ctrl**?
2. ¿Por qué un sistema que utilice dos o más niveles de privilegio diferentes debe usar una TSS aún cuando la conmutación de tareas es manual?